

Lecture-#6

Natural Language Processing

A8706

Introduction

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

2.2. Methods

- ❖ Sentence segmentation and topic segmentation have been considered as a **boundary classification problem**.
- ❖ Given a boundary candidate(between two word tokens for sentence segmentation and between two sentences for topic segmentation), the goal is to predict whether or not the candidate is an actual boundary (sentence or topic boundary).
- ❖ Formally, let $\mathbf{x} \in \mathcal{X}$ be the **vector of features** (the observation) associated with a candidate and $\mathbf{y} \in \mathcal{Y}$ be the **label** predicted for that candidate.
- ❖ The label y can be **b for boundary** and **b' for non boundary**.
 - ❖ Classification problem: given a set of training examples $(x, y)_{\text{train}}$, find a function that will assign the most accurate possible label y of unseen examples x unseen.
 - ❖ Alternatively to the binary classification problem, it is possible to model boundary types using finer-grained categories.
 - ❖ For segmentation in text be framed as a three-class problem: sentence boundary b^a , without an abbreviation $b^{a'}$ and abbreviation not as a boundary b^{-a}

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

It is also possible to see the candidate boundaries as a sequence and search for the sequence of boundary types $\hat{Y} = (\hat{y}_1, \dots, \hat{y}_n)$ that have the maximum probability given the candidate examples, $\mathbf{X} = (x_1, \dots, x_n)$:

$$\hat{Y} = \arg \max_y P(Y|X)$$

- We categorize the methods into local and sequence classification.
- Another categorization of methods is done according to the type of the machine learning algorithm: **generative versus discriminative**.
- Generative sequence models estimate the **joint distribution** of the observations $P(X, Y)$ (words, punctuation) and the labels (sentence boundary, topic boundary).
- Discriminative sequence models, however, **focus on features** that categorize the differences between the labelling of that example.

- $\hat{y}_i$: The predicted label for the input x_i .
- $P(y_i | x_i)$: The posterior probability of label y_i given the input x_i .
- $\arg \max_{y_i \in \mathcal{Y}}$: The operation that finds the value of y_i in the set of possible labels $\mathcal{Y}$ that maximizes the posterior probability.

The posterior probability is given by Bayes' theorem:

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y) \cdot P(y)}{P(\mathbf{x})}$$

Where:

- $P(y|\mathbf{x})$: The posterior probability of label y given the data $\mathbf{x}$.
- $P(\mathbf{x}|y)$: The likelihood, which is the probability of the data $\mathbf{x}$ given that the label is y .

Example: Spam Email Classification

Suppose you are working on a spam email classifier. The goal is to classify whether an email is "spam" or "not spam" based on its content.

1.Input (x_i): Features extracted from the email text, like the frequency of certain words (e.g., "free," "win," "money").

2.Possible Labels ($\mathcal{Y}$): The set of possible labels is $\mathcal{Y} = \{\text{"spam"}, \text{"not spam"}\}$.

3.Posterior Probability ($P(y_i|x_i)$): This is the probability that the email is either "spam" or "not spam" given the features.

The MAP estimate involves calculating the posterior probability for both "spam" and "not spam" labels and then choosing the label with the highest probability.

•For example, given an email with certain words, you might calculate:

- $P(\text{"spam"}|x_i) = 0.8$
- $P(\text{"not spam"}|x_i) = 0.2$

4.Prediction ($\hat{y}_i$): The classifier will predict "spam" because it has the higher posterior probability.

Thus, the formula $\hat{y}_i = \arg \max_{(y_i \in \mathcal{Y})} P(y_i|x_i)$ in this context would output "spam" as the predicted label.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Generative Sequence Classification Methods:

- ❖ Generative sequence classification methods are a class of machine learning techniques that combine elements of sequence generation and classification tasks.
- ❖ These methods aim to generate sequences while also assigning a class label or category to each sequence.
- ❖ This can be particularly useful in various natural language processing (NLP) and bioinformatics tasks, among others.
- ❖ Generative sequence classification methods find applications in various domains, including natural language processing, bioinformatics, speech recognition, and more.
- ❖ The choice of method depends on the specific task and dataset, as well as the available computational resources and desired model performance.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- A Hidden Markov Model (HMM) is a statistical model that represents systems where the state is partially observable. It's widely used in various applications such as speech recognition, natural language processing, and bioinformatics.

- **Components of an HMM**

- 1. States:**

- These are the possible conditions or situations that the system can be in. However, in an HMM, these states are hidden, meaning you cannot directly observe them.
- Example: In a speech recognition system, the states could represent different phonemes (basic units of sound).

- 2. Observations:**

- These are the data or evidence that we can observe and are influenced by the underlying hidden states.
- Example: In speech recognition, the observations could be the acoustic signals or features extracted from audio.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

3. Transition Probabilities (A):

- These probabilities represent the likelihood of moving from one state to another.

$$a_{ij} = P(S_{t+1} = j | S_t = i)$$

- is the probability of transitioning from state i to state j

4. Emission Probabilities (B):

- These are the probabilities of observing a particular observation given a specific state.

$$b_j(o_t) = P(O_t = o | S_t = j)$$

is the probability of observing o given the system is in state j.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

5. Initial Probabilities (π):

$$\pi_i = P(S_1 = i)$$

These are the probabilities of the system starting in a particular state.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- **Example: Part-of-Speech Tagging**

- Imagine you're working on tagging words in a sentence with their corresponding part-of-speech (POS) tags (e.g., noun, verb).
- **States:** The possible POS tags (e.g., Noun, Verb, Adjective).
- **Observations:** The words in the sentence.
- **Transition Probabilities:** The probability of moving from one POS tag to another.
- **Emission Probabilities:** The probability of a particular word being associated with a particular POS tag.
- **Initial Probabilities:** The probability distribution over POS tags for the first word in the sentence.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Generative Sequence Classification Methods:

Generative models, to find the conditional probability $P(Y|X)$, they estimate the prior probability $P(Y)$ and likelihood probability $P(X|Y)$ with the help of the training data and use the Bayes Theorem to calculate the posterior probability $P(Y|X)$:

$$\text{posterior} = \frac{\text{prior} \times \text{likelihood}}{\text{evidence}} \Rightarrow P(Y|X) = \frac{P(Y) \cdot P(X|Y)}{P(X)}$$

$P(X)$ in the denominator is dropped because it is fixed for different Y and hence does not change the argument of max. $P(X|Y)$, and $P(Y)$ can be estimated as

$$P(X|Y) = \prod_{i=1}^n P(\mathbf{x}_i | y_1, \dots, y_i)$$

$$P(Y) = \prod_{i=1}^n P(y_i | y_1, \dots, y_{i-1})$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Simplifying assumptions can be made to make the computation of these probabilities tractable:

$$P(\mathbf{x}_i | y_1, \dots, y_i) \approx P(\mathbf{x}_i | y_i)$$

and a bigram model can be assumed for modeling output categories:

$$P(y_i | y_1, \dots, y_{i-1}) \approx P(y_i | y_{i-1})$$

Finding the Structure of Words and Documents

Model for the two-class problem, for example *nonboundary* (NB) and *sentence boundary* (SB) for sentence segmentation.

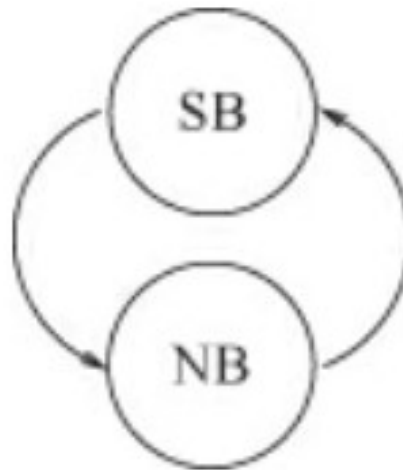


Fig: Conceptual hidden Markov model for segmentation with two states: one for segment boundaries, one for others

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

<i>Emitted Words</i>	...	people	are	dead	few	pictures	...
<i>State Sequence</i>	...	NB	NB	SB	NB	NB	...

Sentence segmentation with simple two-state Markov model

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Mr. Smith went to Washington. He had a meeting at 10 a.m. It was rescheduled.

Annotated with SB and NB:

1. "Mr. Smith went to Washington."

1. "Mr." → **NB** (Non-boundary, because "Mr." is an abbreviation, not the end of a sentence)

2. "Washington." → **SB** (Sentence Boundary, marking the end of a sentence)

2. "He had a meeting at 10 a.m."

1. "a.m." → **NB** (Non-boundary, because "a.m." is an abbreviation, not the end of a sentence)

2. "a.m." → **SB** (Sentence Boundary, marking the end of a sentence after "a.m.")

3. "It was rescheduled."

1. "rescheduled." → **SB** (Sentence Boundary, marking the end of a sentence)

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Mr. Smith went to Washington. He had a meeting at 10 a.m. It was rescheduled.

- **Non-boundary (NB):** This is where a period (or other punctuation) does not indicate the end of a sentence. Common examples include abbreviations like "Mr.," "Dr.," "a.m.," or "etc."

- **Examples:**

- "Mr. Smith" → **NB**

- "10 a.m." → **NB**

- **Sentence Boundary (SB):** This is where a period (or other punctuation) actually marks the end of a sentence.

- **Examples:**

- "Washington." → **SB**

- "a.m." (when followed by a capitalized word or new sentence) → **SB**

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Generative models are considered a class of statistical models that can generate new data instances. These models are used in unsupervised machine learning as a means to perform tasks such as:

- ❖ Probability and Likelihood estimation,
- ❖ Modeling data points
- ❖ To describe the phenomenon in data,
- ❖ To distinguish between classes based on these probabilities.

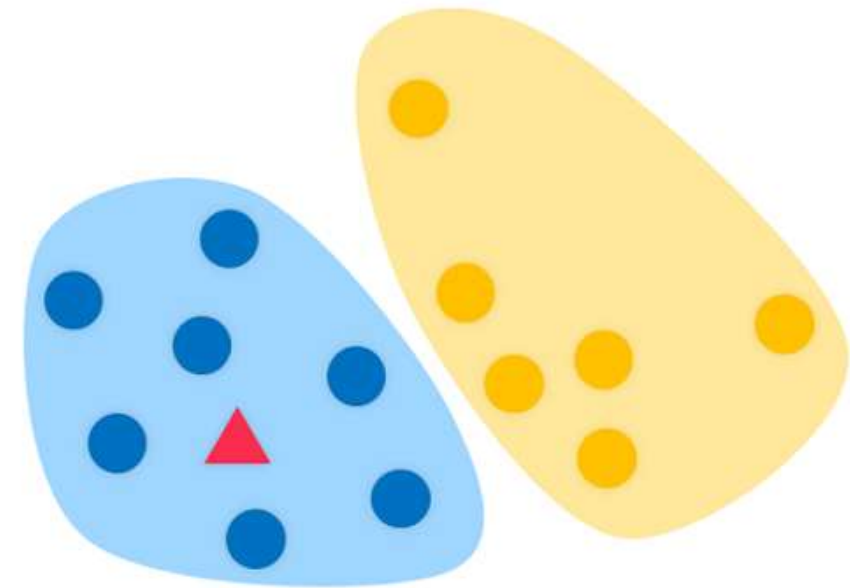
Since these models often rely on the Bayes theorem to find the joint probability, generative models can tackle a more complex task than analogous discriminative models.

Finding the Structure of Words and Documents

❖ The Generative approach focuses on the distribution of individual classes in a dataset, and the learning algorithms tend to model the underlying patterns or distribution of the data points (e.g., gaussian). These models use the concept of joint probability and create instances where a given **feature (x)** or input and the desired output or **label (y)** exist simultaneously.

❖ These models use **probability estimates** and **likelihood** to model data points and differentiate between different class labels present in a dataset. Unlike discriminative models, these models can also generate new data points.

Generative



Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Training generative classifiers involve estimating a function $f: X \rightarrow Y$, or probability $P(Y|X)$:

- ❖ Assume some functional form for the probabilities such as $P(Y)$, $P(X|Y)$
- ❖ With the help of training data, we estimate the parameters of $P(X|Y)$, $P(Y)$
- ❖ Use the Bayes theorem to calculate the posterior probability $P(Y|X)$

Examples of Generative Models

- ❖ Naïve Bayes
- ❖ Bayesian networks
- ❖ Markov random fields
- ❖ Hidden Markov Models (HMMs)
- ❖ Latent Dirichlet Allocation (LDA)
- ❖ Generative Adversarial Networks (GANs)
- ❖ Autoregressive Model

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Discriminative Local Classification Methods:

- ❖ Discriminative local classification methods are a class of machine learning techniques that focus on making decisions or predictions for individual data points within a **larger dataset**.
- ❖ These methods aim to classify data points into specific categories or classes based on their features while considering the local context around each data point.
- ❖ The discriminative model refers to a class of models used in **Statistical Classification**, mainly used for supervised machine learning.
- ❖ These types of models are also known as **conditional models** since they learn the boundaries between classes or labels in a dataset.
- ❖ These discriminative local classification methods are used in a wide range of applications, including **image classification**, **text classification**, **anomaly detection**, and more.
- ❖ The choice of method depends on the nature of the data, the problem at hand, and the trade-offs between **model complexity** and **performance**.

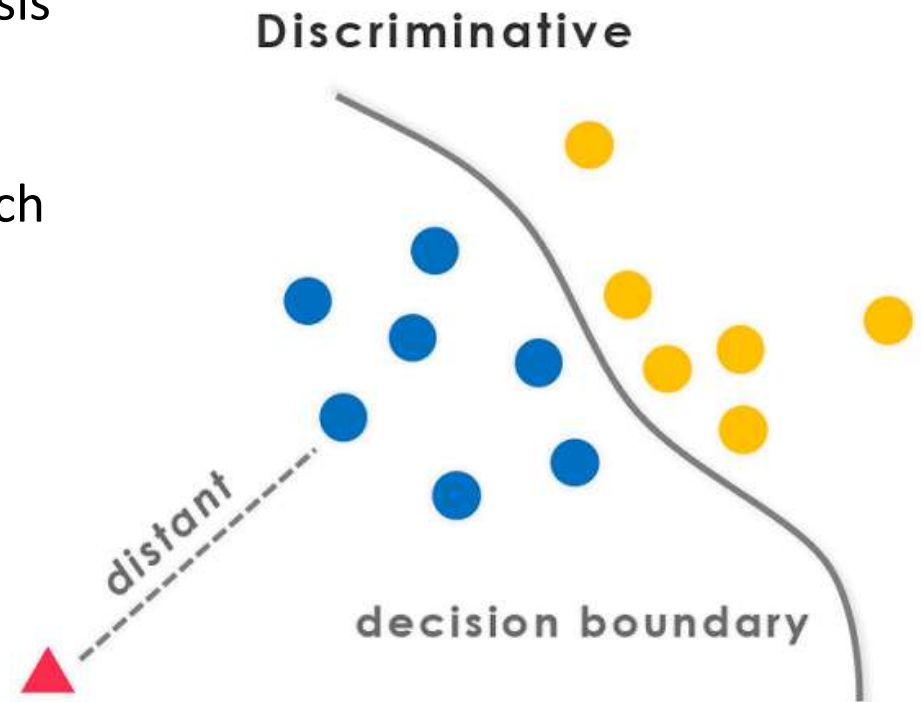
Finding the Structure of Words and Documents

Training discriminative classifiers or discriminant analysis involves estimating a function $f: X \rightarrow Y$, or probability $P(Y|X)$

- ❖ Assume some functional form for the probability, such as $P(Y|X)$
- ❖ With the help of training data, we estimate the parameters of $P(Y|X)$

Examples of Discriminative Models

- ❖ Logistic regression
- ❖ Support vector machines(SVMs)
- ❖ Traditional neural networks
- ❖ Nearest neighbor
- ❖ Conditional Random Fields (CRFs)
- ❖ Decision Trees and Random Forest



Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

•Local Classification:

- In structured prediction tasks, such as **part-of-speech tagging** or **named entity recognition**, a local classifier is applied to each element (**word, token**) in the sequence to make a prediction.
- The prediction for each element is made independently, often based on a window of surrounding elements (the local context).

•Discriminative Models:

- Discriminative models are those that model the decision boundary directly. Instead of modeling the joint probability of the inputs and outputs, they model the **conditional probability $P(y|x)$**

•Structured Prediction:

- Structured prediction refers to tasks where the output is not a single label, but a structured object like a sequence or a tree. Examples include **sequence labeling tasks**, **parsing**, and **machine translation**.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

•Advantages of Discriminative Local Classification

- **Flexibility:** Discriminative models can directly incorporate a variety of features, including non-independent features, making them more flexible than generative models.
- **Efficiency:** Local classifiers can be computationally efficient because they make predictions based on local information, rather than modeling the entire sequence globally.
- **Scalability:** These methods can scale well to large datasets and complex tasks by focusing on local decisions.

•Disadvantages

- **Error Propagation:** Since decisions are made locally, errors made early in the sequence can propagate, leading to errors later in the sequence.
- **Lack of Global Context:** These methods may miss global dependencies that could be captured by global models or structured prediction approaches like CRFs.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

The quick brown fox jumps over the lazy dog.

Steps Involved

1. Feature Extraction

For each word in the sentence, we extract features from a window of words around it. These features could include:

- The word itself
- Previous and next words
- Capitalization (is the word capitalized?)
- Suffixes or prefixes of the word
- Part-of-speech tags of previous words (if already predicted)
- Position in the sentence

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

The quick brown fox jumps over the lazy dog.

2. Local Classification

We use a discriminative model (e.g., logistic regression, SVM, or a neural network) to predict the POS tag for each word based on the extracted features.

Let's consider the first few words:

Word: "The"

Features:

Current word: "The"

Next word: "quick"

Capitalization: Yes

Prefixes/Suffixes: "Th-", "-he"

Word: "quick"

Features:

Previous word: "The"

Current word: "quick"

Next word: "brown"

Capitalization: No

Prefixes/Suffixes: "qui-", "-ick"

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

The quick brown fox jumps over the lazy dog.

3. Prediction

The model makes a prediction for each word:

"The" → DET (Determiner)

"quick" → ADJ (Adjective)

"brown" → ADJ (Adjective)

"fox" → NOUN (Noun)

"jumps" → VERB (Verb)

"over" → PREP (Preposition)

"the" → DET (Determiner)

"lazy" → ADJ (Adjective)

"dog" → NOUN (Noun)

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Two Methods for computing the similarity scores

- block comparison
- vocabulary introduction

Given two blocks, b_1 and b_2 , each having k tokens (sentences or paragraphs), the similarity (or topical cohesion) score is computed by the formula:

$$\frac{\sum_t w_{t,b_1} \cdot w_{t,b_2}}{\sqrt{\sum_t w_{t,b_1}^2 \sum_t w_{t,b_2}^2}}$$

where $w_{t,b}$ is the weight assigned to term t in block b . The weights can be binary or may be computed using other information retrieval-based metrics such as term frequency.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Assigns a score to a token-sequence gap on the basis of how many new words are seen in the interval in which it is the midpoint. Similar to the block comparison formulation, given two consecutive blocks, b_1 and b_2 , of equal number of words, w , the topical cohesion score is computed with the following formula:

$$\frac{NumNewTerms(b_1) + NumNewTerms(b_2)}{2 \times w}$$

where $NumNewTerms(b)$ returns the number of terms in block b , seen for the first time in text.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Discriminative Sequence Classification Methods:

- ❖ In segmentation tasks, the sentence or topic decision for a given example(word, sentence, paragraph) highly depends on the decision for the examples in its vicinity.
- ❖ Discriminative sequence classification methods are in general extensions of local discriminative models with additional decoding stages that find the best assignment of labels by looking at neighboring decisions to label.
- ❖ Conditional random fields(CRFs) are extension of maximum entropy, SVM struct is an extension of SVM, and maximum margin Markov networks(M³N) are extensions of HMM.
- ❖ CRFs are a class of log-linear models for labelling structures.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- They Predict sentence or topic boundaries independently, CRFs can oversee the whole sequence of boundary Hypotheses to make their decisions.
- Formally, they model the conditional probability of a sequence of boundary labels ($Y = y_1, \dots, y_n$) given the sequence of feature sets extracted from the context in which they occur ($X = \mathbf{x}_1, \dots, \mathbf{x}_n$).

$$P(Y|X) \sim \frac{1}{Z(X)} \exp \left(\sum_{t=1}^n \sum_{i=1}^m \lambda_i f_i(y_{t-1}, y_t, y_t) \right)$$

$$Z(X) = \sum_Y \exp \left(\sum_{t=1}^n \sum_{i=1}^m \lambda_i f_i(y_{t-1}, y_t, y_t) \right)$$

where $f_i(\cdot)$ are feature functions of the observations and a clique of labels, and λ_i are the corresponding weights. $Z(\cdot)$ is a normalization function dependent only on the observations.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

1. Sequence Classification:

- Unlike local classification, which makes predictions independently for each element, sequence classification considers the entire sequence of inputs when making predictions.
- The goal is to assign a label to each element in the sequence, taking into account the dependencies between neighbouring elements.

2. Discriminative Models:

- Discriminative models focus on directly estimating the conditional probability $P(y \mid x)$ where x is the sequence of inputs and y is the sequence of labels.
- These models are trained to optimize the decision boundary between different classes based on the input features.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

3. Common Discriminative Sequence Models:

- **Conditional Random Fields (CRFs):** One of the most popular discriminative models for sequence classification. CRFs model the conditional probability of the entire sequence of labels given the sequence of inputs.
- **Structured Support Vector Machines (SVMs):** These extend the SVM framework to handle structured outputs, including sequences.
- **Recurrent Neural Networks (RNNs):** RNNs, including Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), are powerful models for handling sequences because they can maintain a memory of previous inputs in the sequence.
- **Transformer-based Models:** Models like BERT and GPT, which rely on the transformer architecture, are also discriminative and can handle sequence prediction by leveraging attention mechanisms to capture long-range dependencies.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

John Doe works at OpenAI in San Francisco.

Using Conditional Random Fields (CRFs)

1. Feature Extraction

For each word in the sentence, extract features such as:

- The word itself.
- Word shape (capitalization, digits).
- Part-of-speech (POS) tags.
- Contextual words (words before and after the current word).

2. Modeling with CRFs

- Sequence Model:** CRFs will model the conditional probability of the sequence of labels (e.g., "B-PER", "I-PER", "O", "B-ORG", "O", "B-LOC", "I-LOC") given the input sequence (the words in the sentence).
- Dependency Capture:** CRFs capture dependencies between neighboring labels, meaning the model understands that if a word is labeled "B-PER" (beginning of a person's name), the next word is likely "I-PER" (inside a person's name) if it's part of the same entity.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

John Doe works at OpenAI in San Francisco.

3. Training

- The CRF model is trained on annotated data, where each word in the sequence is labeled with its corresponding named entity tag.
- During training, the model learns to maximize the conditional likelihood of the correct sequence of labels given the input sequence.

4. Inference

- Given a new sentence, the CRF model will predict the most likely sequence of tags that fits the entire sequence of words, taking into account both the individual word features and the transition probabilities between tags.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Advantages of Discriminative Sequence Models

- **Global Sequence Consideration:** These models consider the entire sequence, which helps capture dependencies between labels that local classification methods might miss.
- **Flexibility in Feature Usage:** Discriminative models can directly incorporate complex and rich features, including non-independent features.
- **Accuracy:** By focusing on the decision boundary, discriminative models often provide better performance in practice, especially in complex tasks.

Disadvantages

- **Complexity:** These models can be more complex to train and require more computational resources, especially with large datasets.
- **Error Sensitivity:** While these models capture dependencies, they can also be sensitive to errors in the input features or label dependencies, potentially leading to cascading errors.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- Nonsequential discriminative classification algorithms typically ignore the context, which is critical for the segmentation task.
- While we may add context as a feature or simply use CRFs, which inherently consider context, these approaches are suboptimal when dealing with real-valued features, such as pause duration or pitch range.
- Most earlier studies simply tackled this problem by binning the feature space either manually or automatically

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- The main idea is to use the posterior probabilities, $P_c(y_i|\mathbf{x}_i)$, for each boundary candidate, obtained from the other classifiers, such as boosting or CRF, by simply converting them to state observation likelihoods by dividing to their priors following the well-known Bayes rule:

$$\operatorname{argmax}_{y_i} \frac{P_c(y_i|\mathbf{x}_i)}{P(y_i)} = \operatorname{argmax}_{y_i} P(\mathbf{x}_i|y_i)$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- Applying the Viterbi algorithm to the HMM then returns the most likely segmentation. To handle dynamic ranges of state transition probabilities and observation likelihoods, a weighting scheme as is usually described in the literature can be applied:

$$\operatorname{argmax}_{y_i} P_c(\mathbf{x}_i|y_i)^\alpha \times P(y_i)^\beta$$

where $P(y)$ is estimated by the HELM, and α and β are optimized using a held-out set.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- Applying the Viterbi algorithm to the HMM then returns the most likely segmentation. To handle dynamic ranges of state transition probabilities and observation likelihoods, a weighting scheme as is usually described in the literature can be applied:

$$\operatorname{argmax}_{y_i} P_c(x_i|y_i)^\alpha \times P(y_i)^\beta$$

where $P(y)$ is estimated by the HELM, and α and β are optimized using a held-out set.

- The **hold-out method** involves splitting the data into multiple parts and **using** one part for training the model and the rest for validating and testing it.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- **HELM** (the hidden event language model)

- In this model, one state is reserved for each boundary token, *SB* and *NB*, and the rest of the states are for generating words. To ease the computation, an imaginary token is inserted between all consecutive words in case the word preceding the boundary is not part of a disfluency.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- ❖ Hybrid approaches in Natural Language Processing (NLP) classifications refer to techniques that combine multiple methods or models to improve the performance of NLP classification tasks.
- ❖ These methods often leverage the strengths of different approaches to address the limitations of individual models. Here are some common hybrid approaches in NLP classifications:

Ensemble Methods:

- ❖ Ensemble methods combine predictions from multiple base models to make a final classification decision.
- ❖ Bagging (Bootstrap Aggregating) methods like Random Forests combine multiple decision trees.
- ❖ Boosting methods like AdaBoost and Gradient Boosting combine weak learners to create a strong classifier.
- ❖ These ensemble methods can improve classification accuracy and robustness.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

Feature Combination:

- ❖ Hybrid models can combine features from different sources or representations to enhance classification performance.
- ❖ For example, combining word embeddings with hand-crafted features or syntactic features can capture both semantic and structural information.

Multi-Modal Fusion:

- ❖ In some NLP tasks, information comes from multiple modalities, such as text, images, or audio.
- ❖ Hybrid models can fuse information from these modalities to make better classifications. For instance, combining textual and visual information for sentiment analysis of multimedia content.

Neural Network Ensembles:

- ❖ Combining multiple neural networks with different architectures or initialization weights can lead to improved performance.
- ❖ Techniques like stacking or blending neural network predictions can be effective in hybrid approaches.

Transfer Learning and Pretraining:

- ❖ Transfer learning models like BERT, GPT, and RoBERTa have become popular in NLP.
- ❖ Hybrid models can fine-tune these pretrained models for specific classification tasks, achieving state-of-the-art results with less training data.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

Rule-Based and Machine Learning Hybrid:

- ❖ Combining rule-based systems with machine learning models can leverage domain knowledge while benefiting from data-driven approaches.
- ❖ For instance, using regular expressions or handcrafted rules to preprocess text data before feeding it into a machine learning classifier.

Hierarchical Approaches:

- ❖ Hybrid models can use a hierarchy of classifiers, where higher-level classifiers make decisions based on the outputs of lower-level classifiers.
- ❖ This approach is useful for complex classification tasks, where different aspects of the problem can be tackled independently.

Sequential and Tree-Based Models:

- ❖ Hybrid models can combine sequential models like Recurrent Neural Networks (RNNs) with tree-based models like Random Forests or Gradient Boosting.
- ❖ This approach can be useful for problems that involve both sequential and non-sequential information.
- Hybrid approaches in NLP classifications are often problem-specific and depend on the nature of the data and the task. They offer flexibility and the potential for significant performance improvements by leveraging the complementary strengths of different techniques and models.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Extensions for Global Modeling for Sentence Segmentation:

- ❖ In Natural Language Processing (NLP), sentence segmentation, also known as sentence boundary detection or sentence tokenization, is the process of identifying and splitting a text into individual sentences.
- ❖ While basic sentence segmentation methods often rely on simple heuristics or punctuation cues, more advanced techniques employ global modeling and extensions to improve accuracy and handle complex cases.
- ❖ These extensions for global modeling in sentence segmentation aim to handle various linguistic complexities, including abbreviations, quotations, nested structures, and different writing styles.
- ❖ The choice of method and extension depends on the specific requirements of the NLP task and the characteristics of the text data being processed.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Complexity of the Approaches

- ❖ In Natural Language Processing (NLP), the complexity of approaches refers to the computational, algorithmic, and resource requirements associated with different methods and models used for various NLP tasks.
- ❖ The complexity can vary significantly depending on the specific approach, the size of the dataset, the complexity of the language, and the available computational resources.

Model Architecture:

The choice of model architecture plays a significant role in complexity. For example, simple models like logistic regression have lower computational demands compared to deep neural networks such as Transformers.

Model Size:

Larger models with more parameters tend to be more complex. Pretrained models like GPT-3 and BERT, which have hundreds of millions or even billions of parameters, require substantial computational resources.

Training Data Size:

The amount of training data affects the complexity of model training. Larger datasets often require more computational power and time to train effectively.

Hyper parameter Tuning:

Finding the optimal hyper parameters for a model can involve a time-consuming and computationally intensive search, especially for complex models.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Tokenization and Sequence Length:

Tokenization, or splitting text into smaller units (e.g., words or subwords), can impact model complexity. Handling long sequences may require special techniques or models designed for long-range dependencies.

Data Preprocessing:

Data preprocessing steps such as text cleaning, feature extraction, and handling imbalanced datasets can add complexity to an NLP pipeline.

Fine-Tuning:

Fine-tuning pretrained models for specific tasks can be complex, as it involves adjusting model weights on a task-specific dataset while retaining knowledge from the pretrained model.

Decoding:

In sequence generation tasks like text summarization or machine translation, decoding the output sequence can be computationally intensive, especially for beam search or sampling-based methods.

Parallelization:

Parallelizing model training and inference across multiple GPUs or TPUs can mitigate complexity by reducing the time required for computation.

Resource Requirements:

Complex models often demand powerful hardware resources, including GPUs or TPUs, to achieve reasonable training and inference times.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Interpretability:

As models become more complex, their interpretability may decrease. Understanding and interpreting the decisions made by complex models can be challenging.

Scaling Up:

To achieve state-of-the-art performance, researchers often resort to scaling up models, which can lead to increased complexity and resource requirements.

Regularization:

To prevent overfitting in complex models, regularization techniques may be needed, adding complexity to the training process.

Deployment Considerations:

Deploying complex NLP models in real-world applications may require optimizations for latency, memory usage, and scalability.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Performance of approaches:

- ❖ The performance of approaches in Natural Language Processing (NLP) can vary significantly depending on the specific task, the choice of algorithms, models, and data.
- ❖ NLP tasks encompass a wide range of applications, and performance is typically evaluated using various metrics tailored to each task.
- ❖ Here are some key performance metrics and factors influencing the performance of NLP approaches:
 - ❖ **Accuracy:** Accuracy measures the proportion of correctly predicted instances in classification tasks. While accuracy is a common metric, it may not be appropriate for imbalanced datasets.
 - ❖ **Precision, Recall, and F1-Score:** Precision measures the fraction of true positives among predicted positives, while recall measures the fraction of true positives among actual positives. The F1-score is the harmonic mean of precision and recall, providing a balanced measure of performance.
 - ❖ **Mean Squared Error (MSE):** MSE is often used to evaluate regression tasks, measuring the average squared difference between predicted and actual values.

1. Common Evaluation Metrics

- **Error Rate:**
 - The ratio of the number of errors to the total number of examples.
- **F-measure (or F1 score):**
 - The harmonic mean of **precision** and **recall**:
 - **Recall**: Proportion of actual sentence boundaries correctly identified.
 - **Precision**: Proportion of identified boundaries that are correct.
- **NIST Error Rate:**
 - Number of incorrectly labeled candidates divided by actual boundaries.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

2. Performance in Text

- **Dataset:** Wall Street Journal Corpus (~27,000 sentences).
- **Results:**
 - **Mikheev's rule-based system:**
 - Error rate: **1.41%**
 - **Adding an abbreviation list:**
 - Error rate drops to **0.45%**
 - **Combining with a supervised classifier (using POS tags):**
 - Error rate further decreases to **0.31%**
 - **Gillick's SVM-based system (no handcrafted rules or abbreviation list):**
 - Error rate: **0.25%**
- **Implication:**
 - Even small error rates are important, as mistakes in sentence segmentation affect all downstream NLP tasks and user-facing outputs.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

3. Performance in Speech

• Mandarin Broadcast News (TDT4 Corpus, Doss et al.):

- **MaxEnt classifier:**
 - F-measure: **69.1%**
- **Adaboost:**
 - F-measure: **72.6%**
- **SVMs:**
 - F-measure: **72.7%**
- **Combination via logistic regression:**
 - Can further improve results.

• Turkish Broadcast News (Guz et al.):

- **HELM:**
 - F-measure: **78.2%**
- **fHELM with morphology features:**
 - F-measure: **86.2%**

• English Broadcast News (Zimmerman et al.):

- **Adaboost + HELM:**
 - F-measure: **67.3%**
- **Adaboost alone:**
 - F-measure: **65.5%**

Adaboost:

F-measure: **86.9%**

CRFs:

F-measure: **89.1%**

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

• Precision is a common metric used to evaluate the performance of classification models, especially in scenarios where false positives are particularly undesirable.

• Formula for Precision

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

• **True Positives (TP):** Instances correctly predicted as positive.

• **False Positives (FP):** Instances incorrectly predicted as positive.

• Steps to Calculate Precision

- **Identify True Positives (TP):** Count how many positive observations were correctly predicted.
- **Identify False Positives (FP):** Count how many negative observations were incorrectly predicted as positive.

• **Apply the formula:** Divide the number of true positives by the sum of true positives and false positives.

- **Example**
- Suppose you have a model that makes the following predictions:
- True Positives (TP): 70
- False Positives (FP): 30

$$\text{Precision} = \frac{70}{70 + 30} = \frac{70}{100} = 0.7$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Recall

Step 1: The Original Text

"The cat sat on the mat. It was a lazy afternoon. The dog barked loudly. A bird chirped from the tree."

Step 2: Reference Annotations (Actual Boundaries - Ground Truth)

First, we manually identify where the true sentence boundaries are. These are our "reference" or "gold standard" annotations.

- "The cat sat on the mat**|**"
- "It was a lazy afternoon**|**"
- "The dog barked loudly**|**"
- "A bird chirped from the tree**|**"

In this reference, there are **4 actual sentence boundaries**.

- (Boundary 1: after "mat")
- (Boundary 2: after "afternoon")
- (Boundary 3: after "loudly")
- (Boundary 4: after "tree")

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Recall

Step 3: Automatically Estimated Boundaries (System Output)

Now, let's imagine a hypothetical system processes the text and identifies sentence boundaries.

"The cat sat on the mat**|** It was a lazy afternoon. The dog barked loudly**|** A bird chirped from the tree**|**"

The system identified boundaries at these positions:

- After "mat" (Matches a reference boundary!)
- After "loudly" (Matches a reference boundary!)
- After "tree" (Matches a reference boundary!)

Step 4: Count the Components

1.Number of Correctly Identified Sentence Boundaries (True Positives):

1. The system correctly identified the boundary after "mat".
2. The system correctly identified the boundary after "loudly".
3. The system correctly identified the boundary after "tree".

4. Count = 3

2.Total Number of Actual Sentence Boundaries (from Reference):

1. We counted this in Step 2.

2. Count = 4

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Recall

$$\text{Recall} = \frac{\text{Number of Correctly Identified Sentence Boundaries}}{\text{Total Number of Actual Sentence Boundaries}}$$

$$\text{Recall} = \frac{3}{4} = 0.75 \text{ or } 75\%$$

A Recall of 75% means that our system was able to find 75% of the *true* sentence boundaries present in the text.

In this case, it missed one actual sentence boundary (the one after "afternoon").

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

The **F1 score** is a way to combine two important metrics—**precision** and **recall**—into a single number that balances both.

- Precision**: Of all the boundaries your system guessed, how many were actually correct?
- Recall**: Of all the actual boundaries, how many did your system find?

The F1 score is **the harmonic mean** of precision and recall, and is especially useful if you want a single score that reflects both metrics, particularly when there's a tradeoff between them.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

F1 Score in Sentence Segmentation

"The cat sat on the mat. It was a lazy afternoon. The dog barked loudly. A bird chirped from the tree."

1. Reference (Ground Truth) Sentence Boundaries:

After "mat": Yes

After "afternoon": Yes

After "loudly": Yes

After "tree": Yes

2. System Output (Detected Boundaries)

Let's say the system predicts boundaries at:

- After "mat" (**Correct**)

- After "loudly" (**Correct**)

- After "tree" (**Correct**)

- After "afternoon" (**Correct**)

- After "cat" (**Wrong—False Positive**)

- After "barked" (**Wrong—False Positive**)

So, the system finds **4 correct boundaries** and makes **2 extra (incorrect) boundaries**.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

3. Calculate Precision and Recall

True Positives (TP):

4 (correct boundaries found: "mat", "afternoon", "loudly", "tree")

False Positives (FP):

2 (incorrect boundaries: after "cat" and after "barked")

False Negatives (FN):

0 (all actual boundaries were detected; none missed)

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{4}{4 + 2} = \frac{4}{6} \approx 0.666 \text{ or } 66.6\%$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{4}{4 + 0} = \frac{4}{4} = 1.0 \text{ or } 100\%$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

4. Calculate F1 Score

$$F1 = 2 \times \frac{0.666 \times 1.0}{0.666 + 1.0}$$

$$F1 = 2 \times \frac{0.666}{1.666}$$

$$F1 = 2 \times 0.3996 \approx 0.799 \text{ or } 79.9\%$$

Metric	Value
Precision	66.6%
Recall	100%
F1 Score	79.9%

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

High Recall (100%): The system found all the true boundaries—none were missed.

Moderate Precision (66.6%): About two-thirds of its predicted boundaries were actually correct. The rest were mistakes.

F1 Score (79.9%): This single number summarizes the system's balance between precision and recall.

NOTE:

F1 will only be high if both precision and recall are high. If either is low, the F1 drops, reflecting the tradeoff!

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Features:

- ❖ In Natural Language Processing (NLP), features are essential components used to represent text and speech data for various tasks, including classification, recognition, and generation.
- ❖ Many features can be applied to both text and speech, as both modalities share certain common characteristics. Here are some features that are commonly used for both text and speech in NLP:

Bag of Words (BoW):

- ❖ BoW represents text by creating a vocabulary of unique words in the corpus and counting their occurrences in each document.
- ❖ For speech, BoW can be applied by transcribing spoken language into text and then using text-based BoW features.
- ❖ **Example:** The sentence "I love cats and dogs" becomes a vector {"I": 1, "love": 1, "cats": 1, "and": 1, "dogs": 1}.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Term Frequency-Inverse Document Frequency (TF-IDF):

- ❖ TF-IDF represents the importance of a term in a document relative to a corpus.
- ❖ It can be used for both text and transcribed speech data to capture the importance of words or terms.
- ❖ Reduces the influence of common words (e.g., "the", "is").
- ❖ **Example:** A word occurring frequently in one document but not others will have a high TF-IDF score.

Word Embeddings:

- Word embeddings like Word2Vec, and FastText represent words in a continuous vector space, capturing semantic relationships.
- Word embeddings can be used for both text and transcribed speech data to capture word-level semantics.
- Word embeddings place similar words like "king" and "queen" close to each other in vector space.

n-grams:

- n-grams represent sequences of n contiguous words or characters.
- They are commonly used in text for capturing local context and can also be applied to transcribed speech data.
- Sequences of n contiguous words or characters. Bigram (2-grams), Trigram (3-grams), etc.
- Captures word patterns and context for better performance in tasks like text classification.
- The sentence "I love cats" can have bigrams like ["I love", "love cats"].

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Part-of-Speech (POS) Tags:

- POS tags represent the grammatical category of each word in a sentence.
- They are used in both text and speech processing to analyze syntactic properties.
- In "The cat runs", cat is tagged as a noun (NN), and runs as a verb (VB).

Phonetic Features:

- For speech data, phonetic features represent acoustic characteristics of speech sounds, such as pitch, duration, and formants (**Formants are frequency peaks in the spectrum which have a high degree of energy. They are especially prominent in vowels. Each formant corresponds to a resonance in the vocal tract**).
- These features can be used for tasks like speech recognition and emotion detection.

Spectrogram Features:

- Spectrogram features represent the time-frequency distribution of audio signals.
- They are used for both speech and environmental sound analysis.

Sentiment Lexicons:

- Sentiment lexicons provide sentiment scores for words.
- They are used in both text and speech sentiment analysis tasks.

Topic Models:

- Topic modeling techniques like Latent Dirichlet Allocation (LDA) can be applied to both text and transcribed speech to identify latent topics in the data.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Syntactic Dependency Features:

Syntactic dependency features represent grammatical relationships between words in a sentence. They are used in both text parsing and spoken language understanding.

Emotion Features:

Emotion features capture emotional content in both text and speech data, helping with tasks like emotion detection or sentiment analysis.

Text-to-Speech (TTS) Features:

TTS features represent acoustic properties of synthesized speech. They are used in TTS systems for generating natural-sounding speech.

Prosodic Features:

Prosodic features capture suprasegmental aspects of speech, such as intonation, rhythm, and stress patterns. They are used in various speech processing tasks, including emotion recognition and speech synthesis.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Processing Stages:

- ❖ Processing stages in Natural Language Processing (NLP) refer to the steps or phases involved in the analysis and understanding of natural language text or speech data.
- ❖ These stages are typically sequential and aim to transform raw language input into structured information that can be used for various NLP tasks.
- ❖ These processing stages form the foundation of many NLP applications, including text classification, machine translation, sentiment analysis, question answering, and more. The choice of stages and techniques depends on the specific NLP task and goals.

Text Acquisition:

The process begins with acquiring raw text or speech data from various sources, such as documents, websites, audio recordings, or chat logs.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Text Preprocessing: In this stage, the raw data is cleaned and standardized. Common preprocessing steps include:

- ❖ Tokenization: Splitting text into words or subword units (e.g., by spaces or punctuation).
- ❖ Lowercasing: Converting all text to lowercase for uniformity.
- ❖ Stop Word Removal: Eliminating common words (e.g., "and," "the") that do not carry significant meaning.
- ❖ Stemming or Lemmatization: Reducing words to their base or root form (e.g., "running" to "run").
- ❖ Spell Correction: Fixing typographical errors and misspellings.
- ❖ Noise Removal: Eliminating irrelevant characters or symbols.

Text Representation: In this stage, text or speech data is transformed into numerical or symbolic representations that machine learning models can work with. Common representations include:

- ❖ Bag of Words (BoW): Representing text as a matrix of word frequencies.
- ❖ Term Frequency-Inverse Document Frequency (TF-IDF): Assigning weights to words based on their importance in documents.
- ❖ Word Embeddings: Representing words as continuous vectors in a vector space.
- ❖ Phonemes: Representing speech as sequences of phonemes, the smallest sound units in a language.

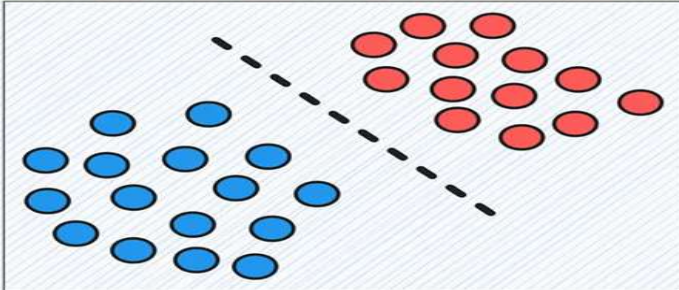
Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Aspect	Generative Models	Discriminative Models
Purpose	Model data distribution	Model conditional probability of labels given data
Use Cases	Data generation, denoising, unsupervised learning	Classification, supervised learning tasks
Common Examples	Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs)	Logistic Regression, Support Vector Machines, Deep Neural Networks
Training Focus	Maximize likelihood of observed data, Capture data structure	Learn decision boundary, Differentiate between classes
Example Task	Image generation, Inpainting (e.g., GANs, VAEs)	Text classification, Object detection (e.g., Deep Neural Networks)

Discriminative Models



Learns the decision boundary between classes

Maximizes the conditional probability: $P(Y|X)$

Directly estimates $P(Y|X)$

Cannot generate new data

Specifically meant for classification tasks

Discriminative models don't possess generative properties

Logistic Regression

Random Forests

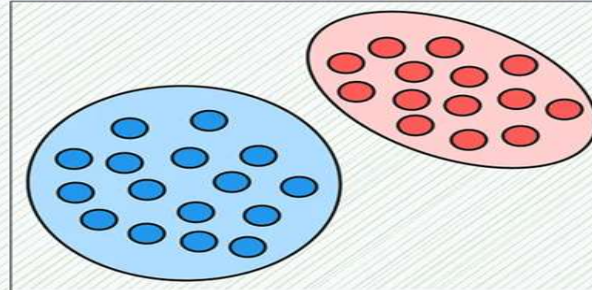
SVMs

Neural Networks

Decision Tree

kNN

Generative Models



Learns the input distribution

Maximizes the joint probability: $P(X, Y)$

Estimates $P(X|Y)$ to find $P(Y|X)$ using Bayes' rule

Can generate new data

Typically, they are NOT used to solve classification tasks

Generative models possess discriminative properties

Hidden Markov Models

Naive Bayes

Gaussian Mixture Models

Gaussian Discriminant Analysis

LDA

Bayesian Networks

